Course Name:	**Database Management system Laboratory**	**Semester:**	**IV**
Date of Performance:	**30/03/2026 (Database was compromised hence late)**	**Batch No:**	**B2**
Faculty Name:	**Prof. Megha Sharma**	**Roll No:**	**16010124107**
Faculty Sign & Date:		**Grade/Marks:**	**25**

Experiment No: 4
Title: DML – select, insert, update and delete

Aim and Objective of the Experiment:

Aim: DML – select, insert, update and delete

1. Nested queries : AND,OR,NOT, IN, NOT IN, Exists, Not Exists, Between, Like, Alias, DISTINCT
2. Update
3. Delete

Objective: To perform various DML Operations and executing nested queries with various clauses.

COs to be achieved:

CO 3: Use SQL for Relational database creation, maintenance and query processing

:

Books/Journals/Websites referred:

1. G. K. Gupta :"*Database Management Systems*", McGraw – Hill
2. Korth, Slberchatz, Sudarshan : "Database Systems Concept", 6th Edition , McGraw Hill
3. Elmasri and Navathe, "Fundamentals *of Database Systems*", 5thEdition, PEARSON Education.

Theory

Select: The SQL **SELECT** statement is used to fetch the data from a database table which returns this data in the form of a result table. These result tables are called result-sets.

Syntax

The basic syntax of the SELECT statement is as follows –

SELECT column1, column2, columnN FROM table_name;

Here, column1, column2... are the fields of a table whose values you want to fetch. If you want to fetch all the fields available in the field, then you can use the following syntax.

SELECT * FROM table_name;

The following code is an example, which would fetch the ID, Name and Salary fields of the customers available in CUSTOMERS table.

SQL> SELECT ID, NAME, SALARY FROM CUSTOMERS;

Insert: The SQL **INSERT INTO** Statement is used to add new rows of data to a table in the database.

Syntax

There are two basic syntaxes of the INSERT INTO statement which are shown below.

INSERT INTO TABLE_NAME (column1, column2, column3,...columnN)

VALUES (value1, value2, value3,...valueN);

Example

The following statements would create record in the CUSTOMERS table.

INSERT INTO CUSTOMERS (ID,NAME,AGE,ADDRESS,SALARY)
VALUES (1, 'Ramesh', 32, 'Ahmedabad', 2000.00);

Update: The SQL **UPDATE** Query is used to modify the existing records in a table. You can use the WHERE clause with the UPDATE query to update the selected rows, otherwise all the rows would be affected.

Syntax:

The basic syntax of the UPDATE query with a WHERE clause is as follows −

UPDATE table_name

SET column1 = value1, column2 = value2...., columnN = valueN

WHERE [condition];

You can combine N number of conditions using the AND or the OR operators.

The following query will update the ADDRESS for a customer whose ID number is 6 in the table.

SQL> UPDATE CUSTOMERS
SET ADDRESS = 'Pune'
WHERE ID = 6;

Delete: The SQL DELETE Query is used to delete the existing records from a table.

You can use the WHERE clause with a DELETE query to delete the selected rows, otherwise all the records would be deleted.

Syntax

The basic syntax of the DELETE query with the WHERE clause is as follows −

DELETE FROM table_name

WHERE [condition];

The following code has a query, which will DELETE a customer, whose ID is 6.

```
SQL> DELETE FROM CUSTOMERS
WHERE ID = 6;
```

Operators used in WHERE are:
= : Equal
> : Greater than
< : Less than
>= : Greater than or equal
<= : Less than or equal
<> : Not equal.

AND, OR and NOT:
- The WHERE clause can be combined with AND, OR, and NOT operators.
- The AND and OR operators are used to filter records based on more than one condition:
- The AND operator displays a record if all the conditions separated by AND are TRUE.
- The OR operator displays a record if any of the conditions separated by OR is TRUE.
- The NOT operator displays a record if the condition(s) is NOT TRUE.

Syntax: SELECT column1, column2, ... FROM table_name WHERE condition1 AND condition2 AND condition3 ...;
SELECT column1, column2, ... FROM table_name WHERE condition1 OR condition2 OR condition3 ...;
SELECT column1, column2, ... FROM table_name WHERE NOT condition;
Example:
SELECT * FROM Customers WHERE Country='India' AND City='Japan';
SELECT * FROM Customers WHERE Country='America' AND (City='India' OR City='Korea');

. Distinct: The SELECT DISTINCT statement is used to return only distinct (different) values.

Syntax: SELECT DISTINCT *column1*, *column2, ...*
FROM *table_name*;

Example: SELECT DISTINCT Country FROM Customers;

LIKE: The LIKE **operator** is used in a WHERE clause to search for a specified pattern in a column.

There are two wildcards used in conjunction with the LIKE operator:

- % - The percent sign represents zero, one, or multiple characters
- _ - The underscore represents a single character

Syntax: SELECT *column1, column2, ...*
FROM *table_name*
WHERE *columnN* LIKE *pattern*

Examples:

1. selects all customers with a CustomerName starting with "a":

SELECT * FROM Customers
WHERE CustomerName LIKE 'a%';

2. selects all customers with a CustomerName that have "r" in the second position:

SELECT * FROM Customers
WHERE CustomerName LIKE '_r%';

IN and Not In: SQL allows testing tuples for membership in a relation. The “in“ connective tests for set membership where the set is a collection of values produced by select clause. The “not in“ connective tests for the absence of set membership. The in and not in connectives can also be used on enumerated sets.

Example:

1. Select fname, mname, lname from employee where designation In (“ceo“,“manager“,“hod“,“assistant“)

2. Select fullname from department where relationship not in(“brother”);

Between: The BETWEEN operator selects values within a given range. The values can be numbers, text, or dates. The BETWEEN operator is inclusive. Begin and end values are included.
Syntax:
SELECT column_name(s)
FROM table_name
WHERE column_name BETWEEN value1 AND value2;
Example:
SELECT * FROM Products WHERE Price BETWEEN 10 AND 20;

Alias: The use of table aliases is to rename a table in a specific SQL statement. The renaming is a temporary change and the actual table name does not change in the database. The column aliases are used to rename a table's columns for the purpose of a particular SQL query.

The basic syntax of a **table** alias is as follows.

SELECT column1, column2....

FROM table_name AS alias_name

WHERE [condition];

The basic syntax of a **column** alias is as follows.

SELECT column_name AS alias_name

FROM table_name

WHERE [condition];

Example:

SELECT C.ID, C.NAME, C.AGE, O.AMOUNT

FROM CUSTOMERS AS C, ORDERS AS O

WHERE C.ID = O.CUSTOMER_ID;

Implementation:
Insert:

INSERT INTO admin VALUES
('IFSC001','Mumbai'),
('IFSC002','Delhi'),
('IFSC003','Pune'),
('IFSC004','Chennai'),
('IFSC005','Kolkata'),

('IFSC006','Bangalore'),
('IFSC007','Hyderabad'),
('IFSC008','Jaipur'),
('IFSC009','Lucknow'),
('IFSC010','Ahmedabad');

	ifsc [PK] character varying (20)	address character varying (60)
1	IFSC001	Mumbai
2	IFSC002	Delhi
3	IFSC003	Pune
4	IFSC004	Chennai
5	IFSC005	Kolkata
6	IFSC006	Bangalore
7	IFSC007	Hyderabad
8	IFSC008	Jaipur
9	IFSC009	Lucknow
10	IFSC010	Ahmedabad

Accounts
INSERT INTO Accounts VALUES
('ACC1',5000,'2024-01-01','Savings','Active','IFSC001'),
('ACC2',6000,'2024-01-02','Current','Active','IFSC002'),
('ACC3',7000,'2024-01-03','Savings','Active','IFSC003'),
('ACC4',8000,'2024-01-04','Current','Active','IFSC004'),
('ACC5',9000,'2024-01-05','Savings','Active','IFSC005'),
('ACC6',10000,'2024-01-06','Savings','Active','IFSC006'),
('ACC7',11000,'2024-01-07','Current','Active','IFSC007'),
('ACC8',12000,'2024-01-08','Savings','Active','IFSC008'),
('ACC9',13000,'2024-01-09','Current','Active','IFSC009'),
('ACC10',14000,'2024-01-10','Savings','Active','IFSC010');

	acc_no [PK] character varying (50)	balance double precision	created_at date	account_type character varying (20)	status character varying (20)	ifsc character varying
1	ACC1	5000	2024-01-01	Savings	Active	IFSC001
2	ACC2	6000	2024-01-02	Current	Active	IFSC002
3	ACC3	7000	2024-01-03	Savings	Active	IFSC003
4	ACC4	8000	2024-01-04	Current	Active	IFSC004
5	ACC5	9000	2024-01-05	Savings	Active	IFSC005
6	ACC6	10000	2024-01-06	Savings	Active	IFSC006
7	ACC7	11000	2024-01-07	Current	Active	IFSC007
8	ACC8	12000	2024-01-08	Savings	Active	IFSC008
9	ACC9	13000	2024-01-09	Current	Active	IFSC009
10	ACC10	14000	2024-01-10	Savings	Active	IFSC010

Employee:
INSERT INTO Employee VALUES
(1,30000,'2023-01-01','Aman','IT','9000000001','Manager','IFSC001'),
(2,25000,'2023-01-02','Riya','HR','9000000002','Exec','IFSC002'),
(3,28000,'2023-01-03','Raj','IT','9000000003','Dev','IFSC003'),
(4,27000,'2023-01-04','Neha','HR','9000000004','Exec','IFSC004'),
(5,35000,'2023-01-05','Karan','IT','9000000005','Lead','IFSC005'),
(6,26000,'2023-01-06','Simran','HR','9000000006','Exec','IFSC006'),
(7,29000,'2023-01-07','Arjun','IT','9000000007','Dev','IFSC007'),
(8,31000,'2023-01-08','Meera','HR','9000000008','Manager','IFSC008'),
(9,32000,'2023-01-09','Vikram','IT','9000000009','Lead','IFSC009'),
(10,24000,'2023-01-10','Sara','HR','9000000010','Exec','IFSC010');

	emp_id [PK] integer	salary integer	hire_date date	name character varying (100)	department character varying (100)	phone_no character varying (15)	designation character varying (100)	ifsc character varying (20)
1	1	30000	2023-01-01	Aman	IT	9000000001	Manager	IFSC001
2	2	25000	2023-01-02	Riya	HR	9000000002	Exec	IFSC002
3	3	28000	2023-01-03	Raj	IT	9000000003	Dev	IFSC003
4	4	27000	2023-01-04	Neha	HR	9000000004	Exec	IFSC004
5	5	35000	2023-01-05	Karan	IT	9000000005	Lead	IFSC005
6	6	26000	2023-01-06	Simran	HR	9000000006	Exec	IFSC006
7	7	29000	2023-01-07	Arjun	IT	9000000007	Dev	IFSC007
8	8	31000	2023-01-08	Meera	HR	9000000008	Manager	IFSC008
9	9	32000	2023-01-09	Vikram	IT	9000000009	Lead	IFSC009
10	10	24000	2023-01-10	Sara	HR	9000000010	Exec	IFSC010

Loans
INSERT INTO LOANS VALUES
('L1',10000,5,'2024-01-01',12,'Active','IFSC001'),
('L2',20000,6,'2024-01-02',12,'Active','IFSC002'),
('L3',15000,5,'2024-01-03',10,'Active','IFSC003'),
('L4',18000,6,'2024-01-04',10,'Active','IFSC004'),
('L5',22000,7,'2024-01-05',12,'Active','IFSC005'),

('L6',25000,6,'2024-01-06',12,'Active','IFSC006'),
('L7',30000,7,'2024-01-07',12,'Active','IFSC007'),
('L8',12000,5,'2024-01-08',8,'Active','IFSC008'),
('L9',17000,6,'2024-01-09',10,'Active','IFSC009'),
('L10',19000,7,'2024-01-10',12,'Active','IFSC010');

Showing rows: 1 to 10 Page No: 1 of 1

	loan_id [PK] character varying (30)	amt double precision	interest double precision	start_date date	duration integer	status character varying (20)	ifsc character varying (20)
1	L1	10000	5	2024-01-01	12	Active	IFSC001
2	L2	20000	6	2024-01-02	12	Active	IFSC002
3	L3	15000	5	2024-01-03	10	Active	IFSC003
4	L4	18000	6	2024-01-04	10	Active	IFSC004
5	L5	22000	7	2024-01-05	12	Active	IFSC005
6	L6	25000	6	2024-01-06	12	Active	IFSC006
7	L7	30000	7	2024-01-07	12	Active	IFSC007
8	L8	12000	5	2024-01-08	8	Active	IFSC008
9	L9	17000	6	2024-01-09	10	Active	IFSC009
10	L10	19000	7	2024-01-10	12	Active	IFSC010

Locker:
INSERT INTO LOCKER VALUES
('LOCK1',2000,'2024-01-01',12,'Active',5,'IFSC001'),
('LOCK2',2100,'2024-01-02',12,'Active',6,'IFSC002'),
('LOCK3',2200,'2024-01-03',12,'Active',7,'IFSC003'),
('LOCK4',2300,'2024-01-04',12,'Active',5,'IFSC004'),
('LOCK5',2400,'2024-01-05',12,'Active',6,'IFSC005'),
('LOCK6',2500,'2024-01-06',12,'Active',7,'IFSC006'),
('LOCK7',2600,'2024-01-07',12,'Active',5,'IFSC007'),
('LOCK8',2700,'2024-01-08',12,'Active',6,'IFSC008'),
('LOCK9',2800,'2024-01-09',12,'Active',7,'IFSC009'),
('LOCK10',2900,'2024-01-10',12,'Active',5,'IFSC010');

	locker_id [PK] character varying (30)	rent double precision	start_date date	duration integer	status character varying (20)	locker_size integer	ifsc character varying (20)
1	LOCK1	2000	2024-01-01	12	Active	5	IFSC001
2	LOCK2	2100	2024-01-02	12	Active	6	IFSC002
3	LOCK3	2200	2024-01-03	12	Active	7	IFSC003
4	LOCK4	2300	2024-01-04	12	Active	5	IFSC004
5	LOCK5	2400	2024-01-05	12	Active	6	IFSC005
6	LOCK6	2500	2024-01-06	12	Active	7	IFSC006
7	LOCK7	2600	2024-01-07	12	Active	5	IFSC007
8	LOCK8	2700	2024-01-08	12	Active	6	IFSC008
9	LOCK9	2800	2024-01-09	12	Active	7	IFSC009
10	LOCK10	2900	2024-01-10	12	Active	5	IFSC010

Locker allocation

```
INSERT INTO LOCKER_ALLOCATION VALUES
('A1','2024-01-01','2025-01-01','Active','LOCK1'),
('A2','2024-01-02','2025-01-02','Active','LOCK2'),
('A3','2024-01-03','2025-01-03','Active','LOCK3'),
('A4','2024-01-04','2025-01-04','Active','LOCK4'),
('A5','2024-01-05','2025-01-05','Active','LOCK5'),
('A6','2024-01-06','2025-01-06','Active','LOCK6'),
('A7','2024-01-07','2025-01-07','Active','LOCK7'),
('A8','2024-01-08','2025-01-08','Active','LOCK8'),
('A9','2024-01-09','2025-01-09','Active','LOCK9'),
('A10','2024-01-10','2025-01-10','Active','LOCK10');
```

	allocation_id [PK] character varying (30)	start_date date	end_date date	status character varying (20)	locker_id character varying (20)
1	A1	2024-01-01	2025-01-01	Active	LOCK1
2	A2	2024-01-02	2025-01-02	Active	LOCK2
3	A3	2024-01-03	2025-01-03	Active	LOCK3
4	A4	2024-01-04	2025-01-04	Active	LOCK4
5	A5	2024-01-05	2025-01-05	Active	LOCK5
6	A6	2024-01-06	2025-01-06	Active	LOCK6
7	A7	2024-01-07	2025-01-07	Active	LOCK7
8	A8	2024-01-08	2025-01-08	Active	LOCK8
9	A9	2024-01-09	2025-01-09	Active	LOCK9
10	A10	2024-01-10	2025-01-10	Active	LOCK10

Customer:

```
INSERT INTO customer VALUES
(1,'Rahul','r1@mail.com','8000000001','111111111111','ACC1','L1','A1'),
(2,'Amit','r2@mail.com','8000000002','222222222222','ACC2','L2','A2'),
(3,'Neha','r3@mail.com','8000000003','333333333333','ACC3','L3','A3'),
(4,'Pooja','r4@mail.com','8000000004','444444444444','ACC4','L4','A4'),
(5,'Raj','r5@mail.com','8000000005','555555555555','ACC5','L5','A5'),
(6,'Karan','r6@mail.com','8000000006','666666666666','ACC6','L6','A6'),
(7,'Simran','r7@mail.com','8000000007','777777777777','ACC7','L7','A7'),
(8,'Arjun','r8@mail.com','8000000008','888888888888','ACC8','L8','A8'),
(9,'Meera','r9@mail.com','8000000009','999999999999','ACC9','L9','A9'),
(10,'Vikram','r10@mail.com','8000000010','101010101010','ACC10','L10','A10');
```

Showing rows: 1 to 10 Page No: 1 of 1

	mer_id integer	name character varying (100)	email character varying (100)	phone_no character varying (15)	aadhar_no character varying (12)	acc_no character varying (50)	loan_id character varying (50)	alloca charac
1	1	Rahul	r1@mail.com	8000000001	111111111111	ACC1	L1	A1
2	2	Amit	r2@mail.com	8000000002	222222222222	ACC2	L2	A2
3	3	Neha	r3@mail.com	8000000003	333333333333	ACC3	L3	A3
4	4	Pooja	r4@mail.com	8000000004	444444444444	ACC4	L4	A4
5	5	Raj	r5@mail.com	8000000005	555555555555	ACC5	L5	A5
6	6	Karan	r6@mail.com	8000000006	666666666666	ACC6	L6	A6
7	7	Simran	r7@mail.com	8000000007	777777777777	ACC7	L7	A7
8	8	Arjun	r8@mail.com	8000000008	888888888888	ACC8	L8	A8
9	9	Meera	r9@mail.com	8000000009	999999999999	ACC9	L9	A9
10	10	Vikram	r10@mail.com	8000000010	101010101010	ACC10	L10	A10

Netbanking

```
INSERT INTO net_banking VALUES
(1,'user1','pass',NOW(),'Active',1),
(2,'user2','pass',NOW(),'Active',2),
(3,'user3','pass',NOW(),'Active',3),
(4,'user4','pass',NOW(),'Active',4),
(5,'user5','pass',NOW(),'Active',5),
(6,'user6','pass',NOW(),'Active',6),
(7,'user7','pass',NOW(),'Active',7),
(8,'user8','pass',NOW(),'Active',8),
(9,'user9','pass',NOW(),'Active',9),
(10,'user10','pass',NOW(),'Active',10);
```

Showing rows: 1 to 10 Page No: 1

	login_id [PK] integer	username character varying (50)	password_hash text	last_login timestamp without time zone	status character varying (20)	customer_id integer
1	1	user1	pass	2026-04-07 11:50:19.172599	Active	1
2	2	user2	pass	2026-04-07 11:50:19.172599	Active	2
3	3	user3	pass	2026-04-07 11:50:19.172599	Active	3
4	4	user4	pass	2026-04-07 11:50:19.172599	Active	4
5	5	user5	pass	2026-04-07 11:50:19.172599	Active	5
6	6	user6	pass	2026-04-07 11:50:19.172599	Active	6
7	7	user7	pass	2026-04-07 11:50:19.172599	Active	7
8	8	user8	pass	2026-04-07 11:50:19.172599	Active	8
9	9	user9	pass	2026-04-07 11:50:19.172599	Active	9
10	10	user10	pass	2026-04-07 11:50:19.172599	Active	10

Payment:

```
INSERT INTO payment VALUES
(1,1000,'UPI','Done',NOW(),1,2),
(2,2000,'Card','Done',NOW(),2,3),
(3,1500,'UPI','Done',NOW(),3,4),
(4,1200,'Cash','Done',NOW(),4,5),
```

(5,1800,'UPI','Done',NOW(),5,6),
(6,2200,'Card','Done',NOW(),6,7),
(7,2500,'UPI','Done',NOW(),7,8),
(8,2700,'Cash','Done',NOW(),8,9),
(9,3000,'UPI','Done',NOW(),9,10),
(10,3500,'Card','Done',NOW(),10,1);

	payment_id [PK] integer	amt double precision	payment_mode character varying (20)	status character varying (20)	timestamp timestamp without time zone	payer_customer_id integer	receiver_customer_id integer
1	1	1000	UPI	Done	2026-04-07 11:50:34.064299	1	2
2	2	2000	Card	Done	2026-04-07 11:50:34.064299	2	3
3	3	1500	UPI	Done	2026-04-07 11:50:34.064299	3	4
4	4	1200	Cash	Done	2026-04-07 11:50:34.064299	4	5
5	5	1800	UPI	Done	2026-04-07 11:50:34.064299	5	6
6	6	2200	Card	Done	2026-04-07 11:50:34.064299	6	7
7	7	2500	UPI	Done	2026-04-07 11:50:34.064299	7	8
8	8	2700	Cash	Done	2026-04-07 11:50:34.064299	8	9
9	9	3000	UPI	Done	2026-04-07 11:50:34.064299	9	10
10	10	3500	Card	Done	2026-04-07 11:50:34.064299	10	1

Transaction:
INSERT INTO transaction VALUES
(1,'ACC1','DEBIT',500,NOW(),1,1),
(2,'ACC2','CREDIT',600,NOW(),2,2),
(3,'ACC3','DEBIT',700,NOW(),3,3),
(4,'ACC4','CREDIT',800,NOW(),4,4),
(5,'ACC5','DEBIT',900,NOW(),5,5),
(6,'ACC6','CREDIT',1000,NOW(),6,6),
(7,'ACC7','DEBIT',1100,NOW(),7,7),
(8,'ACC8','CREDIT',1200,NOW(),8,8),
(9,'ACC9','DEBIT',1300,NOW(),9,9),
(10,'ACC10','CREDIT',1400,NOW(),10,10);

Showing rows: 1 to 10 Page No: 1 of 1

	transaction_id [PK] integer	acc_no character varying (50)	type character varying (10)	amt double precision	timestamp timestamp without time zone	payment_id integer	netbanking_id integer
1	1	ACC1	DEBIT	500	2026-04-07 11:51:05.390154	1	1
2	2	ACC2	CREDIT	600	2026-04-07 11:51:05.390154	2	2
3	3	ACC3	DEBIT	700	2026-04-07 11:51:05.390154	3	3
4	4	ACC4	CREDIT	800	2026-04-07 11:51:05.390154	4	4
5	5	ACC5	DEBIT	900	2026-04-07 11:51:05.390154	5	5
6	6	ACC6	CREDIT	1000	2026-04-07 11:51:05.390154	6	6
7	7	ACC7	DEBIT	1100	2026-04-07 11:51:05.390154	7	7
8	8	ACC8	CREDIT	1200	2026-04-07 11:51:05.390154	8	8
9	9	ACC9	DEBIT	1300	2026-04-07 11:51:05.390154	9	9
10	10	ACC10	CREDIT	1400	2026-04-07 11:51:05.390154	10	10

Alter:

ALTER TABLE customer
ADD city VARCHAR(50);

Showing rows: 1 to 10 Page No: 1 of 1

	(100)	email character varying (100)	phone_no character varying (15)	aadhar_no character varying (12)	acc_no character varying (50)	loan_id character varying (50)	allocation_id character varying (50)	city characte
1		r1@mail.com	8000000001	111111111111	ACC1	L1	A1	[null]
2		r2@mail.com	8000000002	222222222222	ACC2	L2	A2	[null]
3		r3@mail.com	8000000003	333333333333	ACC3	L3	A3	[null]
4		r4@mail.com	8000000004	444444444444	ACC4	L4	A4	[null]
5		r5@mail.com	8000000005	555555555555	ACC5	L5	A5	[null]
6		r6@mail.com	8000000006	666666666666	ACC6	L6	A6	[null]
7		r7@mail.com	8000000007	777777777777	ACC7	L7	A7	[null]
8		r8@mail.com	8000000008	888888888888	ACC8	L8	A8	[null]
9		r9@mail.com	8000000009	999999999999	ACC9	L9	A9	[null]
10		r10@mail.com	8000000010	101010101010	ACC10	L10	A10	[null]

DROP
ALTER TABLE customer
DROP COLUMN city;

Showing rows: 1 to 10 Page No: 1 of 1

	er varying (100)	email character varying (100)	phone_no character varying (15)	aadhar_no character varying (12)	acc_no character varying (50)	loan_id character varying (50)	allocation_id character varying (50)
1		r1@mail.com	8000000001	111111111111	ACC1	L1	A1
2		r2@mail.com	8000000002	222222222222	ACC2	L2	A2
3		r3@mail.com	8000000003	333333333333	ACC3	L3	A3
4		r4@mail.com	8000000004	444444444444	ACC4	L4	A4
5		r5@mail.com	8000000005	555555555555	ACC5	L5	A5
6		r6@mail.com	8000000006	666666666666	ACC6	L6	A6
7		r7@mail.com	8000000007	777777777777	ACC7	L7	A7
8		r8@mail.com	8000000008	888888888888	ACC8	L8	A8
9		r9@mail.com	8000000009	999999999999	ACC9	L9	A9
10		r10@mail.com	8000000010	101010101010	ACC10	L10	A10

Conclusion:

DML operations performed successfully including insert, update, delete, and alter ensuring data integrity, consistency, and effective database manipulation and management

Post Lab Questions:

1. **In SQL, which of the following is not a data Manipulation Language Commands?**

   a) Delete
   b) Truncate
   c) Update
   d) Create

Ans: b

2. **Write SQL query for following statements:**

   a. retrieve all student who his grade has not been awarded
   b. Find the names of all instructors in the Computer Science department
   c. Find the names of all student whose name starts with 'S'.
   d. find the names of instructors with salary amounts between $90,000 and $100,000.

```
-- grade not awarded
SELECT * FROM student
WHERE grade IS NULL;

-- instructors in CS
SELECT name FROM instructor
WHERE department = 'Computer Science';

-- student name starts with S
SELECT name FROM student
WHERE name LIKE 'S%';

-- instructor salary between
SELECT name FROM instructor
WHERE salary BETWEEN 90000 AND 100000;
```